

CS3237 Lab 5 Introduction to Deep Learning

[Student Number | Name | :-----|:-----| | Wira Azmoon Ahmad | | R Ramana |

1. Introduction

We will achieve the following objectives in this lab:

1. An understanding of the practical limitations of using dense networks in complex tasks
2. Hands-on experience in building a deep learning neural network to solve a relatively complex task.

Each step may take a long time to run. You and your partner may want to work out how to do things simultaneously, but please do not miss out on any learning opportunities.

2. Submission Instructions

Please work together as a team of 2 to complete this lab. You will need to submit ONE copy of this notebook per team, but please fill in the names of both team members above. This lab is worth 55 marks.

DO NOT SUBMIT MORE THAN ONE COPY OF THIS LAB!

3. Creating a Dense Network for CIFAR-10

We will now begin building a neural network for the CIFAR-10 dataset. The CIFAR-10 dataset consists of 50,000 32x32x3 (32x32 pixels, RGB channels) training images and 10,000 testing images (also 32x32x3), divided into the following 10 categories:

1. Airplane
2. Automobile
3. Bird
4. Cat
5. Deer
6. Dog
7. Frog
8. Horse
9. Ship
10. Truck

In the first two parts of this lab we will create a classifier for the CIFAR-10 dataset.

3.1 Loading the Dataset

We begin firstly by creating a Dense neural network for CIFAR-10. The code below shows how we load the CIFAR-10 dataset:

```
In [ ]: from tensorflow.keras.utils import to_categorical
        from tensorflow.keras.datasets import cifar10

        def load_cifar10():
            (train_x, train_y), (test_x, test_y) = cifar10.load_data()
            train_x = train_x.reshape(train_x.shape[0], 3072) # Question 1
            test_x = test_x.reshape(test_x.shape[0], 3072) # Question 1
            train_x = train_x.astype('float32')
            test_x = test_x.astype('float32')
            train_x /= 255.0
```

```

test_x /= 255.0
ret_train_y = to_categorical(train_y,10)
ret_test_y = to_categorical(test_y, 10)

return (train_x, ret_train_y), (test_x, ret_test_y)

(train_x, train_y), (test_x, test_y) = load_cifar10()

```

Question 1

Explain what the following two statements do, and where the number "3072" came from (2 MARKS):

```

train_x = train_x.reshape(train_x.shape[0], 3072) # Question 1
test_x = test_x.reshape(test_x.shape[0], 3072) # Question 1

```

They flatten each datapoint to a vector of size 3072. The number of rows is the number of datapoints, and the number of columns is the number pixels in the image, multiplied by 3 colour channels.

FOR GRADER: _____ / 2

3.2 Building the MLP Classifier

In the code box below, create a new fully connected (dense) multilayer perceptron classifier for the CIFAR-10 dataset. To begin with, create a network with one hidden layer of 1024 neurons, using the SGD optimizer. You should output the training and validation accuracy at every epoch, and train for 50 epochs:

```

In [ ]: """
Write your code to build an MLP with one hidden layer of 1024 neurons,
with an SGD optimizer. Train for 50 epochs, and output the training and
validation accuracy at each epoch.
"""

INPUT_FEATS = 3072
N_LAYER_1 = 1024
N_CLASSES = 10

from keras.models import Sequential
from keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.optimizers.schedules import ExponentialDecay

# Create the neural network
nn = Sequential()
nn.add(Dense(N_LAYER_1, input_shape = (INPUT_FEATS, ), activation = 'relu'))
nn.add(Dense(N_CLASSES, activation = 'softmax'))

lr_schedule = ExponentialDecay(initial_learning_rate=1e-1, decay_steps=10000, decay_rate=1)

# Create our optimizer
sgd = SGD(learning_rate = lr_schedule)

# 'Compile' the network to associate it with a loss function,
# an optimizer, and what metrics we want to track
nn.compile(loss='categorical_crossentropy', optimizer=sgd,
           metrics = ['accuracy'])

In [ ]: nn.fit(train_x, train_y, shuffle = True, epochs = 50,
              validation_data = (test_x, test_y))

```

Epoch 1/50
1563/1563 [=====] - 8s 3ms/step - loss: 1.9469 - accuracy: 0.2975 - val_loss: 1.8655 -
val_accuracy: 0.3224
Epoch 2/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.7795 - accuracy: 0.3624 - val_loss: 1.8241 -
val_accuracy: 0.3369
Epoch 3/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.7129 - accuracy: 0.3875 - val_loss: 1.7256 -
val_accuracy: 0.3914
Epoch 4/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.6660 - accuracy: 0.4078 - val_loss: 1.9209 -
val_accuracy: 0.3533
Epoch 5/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.6316 - accuracy: 0.4241 - val_loss: 2.0849 -
val_accuracy: 0.3230
Epoch 6/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.5873 - accuracy: 0.4363 - val_loss: 1.7138 -
val_accuracy: 0.3842
Epoch 7/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.5645 - accuracy: 0.4445 - val_loss: 1.6477 -
val_accuracy: 0.4263
Epoch 8/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.5345 - accuracy: 0.4577 - val_loss: 1.5804 -
val_accuracy: 0.4455
Epoch 9/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.5152 - accuracy: 0.4648 - val_loss: 1.6699 -
val_accuracy: 0.4010
Epoch 10/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.4903 - accuracy: 0.4735 - val_loss: 1.6074 -
val_accuracy: 0.4491
Epoch 11/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.4683 - accuracy: 0.4813 - val_loss: 1.6371 -
val_accuracy: 0.4308
Epoch 12/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.4506 - accuracy: 0.4865 - val_loss: 1.6596 -
val_accuracy: 0.4336
Epoch 13/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.4355 - accuracy: 0.4928 - val_loss: 1.6347 -
val_accuracy: 0.4400
Epoch 14/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.4126 - accuracy: 0.5011 - val_loss: 1.7145 -
val_accuracy: 0.4218
Epoch 15/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.3949 - accuracy: 0.5097 - val_loss: 1.6171 -
val_accuracy: 0.4522
Epoch 16/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.3759 - accuracy: 0.5139 - val_loss: 1.6235 -
val_accuracy: 0.4445
Epoch 17/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.3601 - accuracy: 0.5190 - val_loss: 1.5967 -
val_accuracy: 0.4443
Epoch 18/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.3399 - accuracy: 0.5271 - val_loss: 1.5739 -
val_accuracy: 0.4603
Epoch 19/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.3247 - accuracy: 0.5323 - val_loss: 1.6910 -
val_accuracy: 0.4340
Epoch 20/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.3067 - accuracy: 0.5372 - val_loss: 1.6381 -
val_accuracy: 0.4373
Epoch 21/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.2957 - accuracy: 0.5433 - val_loss: 1.6237 -
val_accuracy: 0.4409
Epoch 22/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.2800 - accuracy: 0.5504 - val_loss: 1.6475 -
val_accuracy: 0.4562
Epoch 23/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.2599 - accuracy: 0.5548 - val_loss: 1.7181 -
val_accuracy: 0.4567
Epoch 24/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.2471 - accuracy: 0.5590 - val_loss: 2.0009 -
val_accuracy: 0.3916

Epoch 25/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.2250 - accuracy: 0.5668 - val_loss: 1.6193 -
val_accuracy: 0.4815
Epoch 26/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.2141 - accuracy: 0.5712 - val_loss: 1.6660 -
val_accuracy: 0.4552
Epoch 27/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.2019 - accuracy: 0.5756 - val_loss: 1.8371 -
val_accuracy: 0.4327
Epoch 28/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.1826 - accuracy: 0.5818 - val_loss: 1.7029 -
val_accuracy: 0.4371
Epoch 29/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.1678 - accuracy: 0.5873 - val_loss: 1.6613 -
val_accuracy: 0.4643
Epoch 30/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.1460 - accuracy: 0.5962 - val_loss: 1.7474 -
val_accuracy: 0.4705
Epoch 31/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.1370 - accuracy: 0.5953 - val_loss: 1.7735 -
val_accuracy: 0.4739
Epoch 32/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.1242 - accuracy: 0.6044 - val_loss: 1.6113 -
val_accuracy: 0.4933
Epoch 33/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.1084 - accuracy: 0.6089 - val_loss: 1.7504 -
val_accuracy: 0.4693
Epoch 34/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.0986 - accuracy: 0.6132 - val_loss: 1.6897 -
val_accuracy: 0.4713
Epoch 35/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.0850 - accuracy: 0.6161 - val_loss: 1.6912 -
val_accuracy: 0.4832
Epoch 36/50
1563/1563 [=====] - 6s 4ms/step - loss: 1.0649 - accuracy: 0.6229 - val_loss: 1.8860 -
val_accuracy: 0.4638
Epoch 37/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.0599 - accuracy: 0.6242 - val_loss: 1.9795 -
val_accuracy: 0.4459
Epoch 38/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.0434 - accuracy: 0.6323 - val_loss: 1.8290 -
val_accuracy: 0.4636
Epoch 39/50
1563/1563 [=====] - 5s 3ms/step - loss: 1.0287 - accuracy: 0.6358 - val_loss: 1.7704 -
val_accuracy: 0.4768
Epoch 40/50
1563/1563 [=====] - 7s 5ms/step - loss: 1.0204 - accuracy: 0.6381 - val_loss: 1.8246 -
val_accuracy: 0.4742
Epoch 41/50
1563/1563 [=====] - 7s 4ms/step - loss: 1.0049 - accuracy: 0.6439 - val_loss: 1.7780 -
val_accuracy: 0.4847
Epoch 42/50
1563/1563 [=====] - 7s 4ms/step - loss: 0.9947 - accuracy: 0.6489 - val_loss: 1.7437 -
val_accuracy: 0.4919
Epoch 43/50
1563/1563 [=====] - 6s 4ms/step - loss: 0.9752 - accuracy: 0.6547 - val_loss: 2.0440 -
val_accuracy: 0.4354
Epoch 44/50
1563/1563 [=====] - 7s 4ms/step - loss: 0.9694 - accuracy: 0.6592 - val_loss: 2.1345 -
val_accuracy: 0.4308
Epoch 45/50
1563/1563 [=====] - 6s 4ms/step - loss: 0.9542 - accuracy: 0.6635 - val_loss: 1.8339 -
val_accuracy: 0.4811
Epoch 46/50
1563/1563 [=====] - 6s 4ms/step - loss: 0.9400 - accuracy: 0.6691 - val_loss: 1.8904 -
val_accuracy: 0.4637
Epoch 47/50
1563/1563 [=====] - 6s 4ms/step - loss: 0.9323 - accuracy: 0.6694 - val_loss: 1.8028 -
val_accuracy: 0.4720
Epoch 48/50
1563/1563 [=====] - 6s 4ms/step - loss: 0.9163 - accuracy: 0.6777 - val_loss: 1.8790 -
val_accuracy: 0.4871

```
Epoch 49/50
1563/1563 [=====] - 5s 3ms/step - loss: 0.9160 - accuracy: 0.6780 - val_loss: 1.8416 -
val_accuracy: 0.4942
Epoch 50/50
1563/1563 [=====] - 5s 3ms/step - loss: 0.8955 - accuracy: 0.6831 - val_loss: 2.0672 -
val_accuracy: 0.4633
```

```
Out[ ]: <keras.callbacks.History at 0x7f50502112d0>
```

Question 2

Complete the following table on the design choices for your MLP (3 MARKS):

Hyperparameter	What I used	Why?	Optimizer	SGD	Specified in question
# of hidden layers	1	Specified in question	# of hidden neurons	1024	Specified in question
Hid layer activation	ReLU	Popular nonlinear function to obtain nonlinear relationship	# of output neurons	10	10 classes
Output activation	Softmax	Obtain probability for each class	learning rate	0.1	Obtains decent results
Default parameters to start with	decay	None	Default parameters to start with	loss	categorical cross entropy
					used for single-label, multi-class classification

FOR GRADER:

Table: ___ / 3

Code: ___ / 5

TOTAL: ___ / 8

Question 3:

What was your final training accuracy? Validation accuracy? Is there overfitting / underfitting? Explain your answer (5 MARKS)

Training: 0.67, Validation: 0.48. There is both overfitting and underfitting simultaneously. There is overfitting as the training accuracy is much larger than the validation accuracy; as well as underfitting as the training accuracy of 0.67 is not very high and could be much higher to fit the train dataset.

FOR GRADER: ___ / 5

3.3 Experimenting with the MLP

Cut and paste your code from Section 3.2 to the box below (you may need to rename your MLP). Experiment with the number of hidden layers, the number of neurons in each hidden layer, the optimization algorithm, etc. See [Keras Optimizers](#) for the types of optimizers and their parameters. **Train for 100 epochs.**

```
In [ ]: """
Cut and paste your code from Section 3.2 below, then modify it to get
much better results than what you had earlier. E.g. increase the number of
nodes in the hidden layer, increase the number of hidden layers,
change the optimizer, etc.

Train for 100 epochs.

"""
"""
Write your code to build an MLP with one hidden layer of 1024 neurons,
with an SGD optimizer. Train for 50 epochs, and output the training and
validation accuracy at each epoch.
"""
INPUT_FEATS = 3072
N_LAYER_1 = 1024
N_LAYER_2 = 512
N_CLASSES = 10

from keras.models import Sequential
from keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.optimizers.schedules import ExponentialDecay

# Create the neural network
```

```

nn2 = Sequential()
nn2.add(Dense(N_LAYER_1, input_shape = (INPUT_FEATS, ), activation = 'sigmoid'))
nn2.add(Dropout(0.2))
nn2.add(Dense(N_LAYER_2, activation = 'sigmoid'))
nn2.add(Dropout(0.2))
nn2.add(Dense(N_CLASSES, activation = 'softmax'))

lr_schedule = ExponentialDecay(initial_learning_rate=1e-1, decay_steps=100000, decay_rate=0.96)

# Create our optimizer
sgd = SGD(learning_rate = lr_schedule, momentum=0.1)

# 'Compile' the network to associate it with a loss function,
# an optimizer, and what metrics we want to track
nn2.compile(loss='categorical_crossentropy', optimizer=sgd,
            metrics = 'accuracy')

```

```

In [ ]: nn2.fit(train_x, train_y, shuffle = True, epochs = 100,
               validation_data = (test_x, test_y))

```

Epoch 1/100
1563/1563 [=====] - 7s 4ms/step - loss: 2.0382 - accuracy: 0.2505 - val_loss: 1.8967 -
val_accuracy: 0.3129
Epoch 2/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.8627 - accuracy: 0.3300 - val_loss: 1.7845 -
val_accuracy: 0.3633
Epoch 3/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.7928 - accuracy: 0.3550 - val_loss: 1.7519 -
val_accuracy: 0.3700
Epoch 4/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.7368 - accuracy: 0.3763 - val_loss: 1.7256 -
val_accuracy: 0.3740
Epoch 5/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.6954 - accuracy: 0.3923 - val_loss: 1.6423 -
val_accuracy: 0.4133
Epoch 6/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.6619 - accuracy: 0.4044 - val_loss: 1.6163 -
val_accuracy: 0.4214
Epoch 7/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.6345 - accuracy: 0.4148 - val_loss: 1.5824 -
val_accuracy: 0.4375
Epoch 8/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.6103 - accuracy: 0.4243 - val_loss: 1.5567 -
val_accuracy: 0.4505
Epoch 9/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.5857 - accuracy: 0.4320 - val_loss: 1.5444 -
val_accuracy: 0.4539
Epoch 10/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.5676 - accuracy: 0.4395 - val_loss: 1.5358 -
val_accuracy: 0.4544
Epoch 11/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.5470 - accuracy: 0.4468 - val_loss: 1.5128 -
val_accuracy: 0.4632
Epoch 12/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.5254 - accuracy: 0.4552 - val_loss: 1.4956 -
val_accuracy: 0.4728
Epoch 13/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.5050 - accuracy: 0.4630 - val_loss: 1.5063 -
val_accuracy: 0.4663
Epoch 14/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.4912 - accuracy: 0.4673 - val_loss: 1.4785 -
val_accuracy: 0.4766
Epoch 15/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.4759 - accuracy: 0.4711 - val_loss: 1.4797 -
val_accuracy: 0.4695
Epoch 16/100
1563/1563 [=====] - 7s 4ms/step - loss: 1.4615 - accuracy: 0.4765 - val_loss: 1.4621 -
val_accuracy: 0.4858
Epoch 17/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.4474 - accuracy: 0.4789 - val_loss: 1.4679 -
val_accuracy: 0.4711
Epoch 18/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.4338 - accuracy: 0.4847 - val_loss: 1.4342 -
val_accuracy: 0.4937
Epoch 19/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.4216 - accuracy: 0.4922 - val_loss: 1.4547 -
val_accuracy: 0.4861
Epoch 20/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.4128 - accuracy: 0.4932 - val_loss: 1.4133 -
val_accuracy: 0.5029
Epoch 21/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.3955 - accuracy: 0.4990 - val_loss: 1.4273 -
val_accuracy: 0.4918
Epoch 22/100
1563/1563 [=====] - 7s 4ms/step - loss: 1.3897 - accuracy: 0.5039 - val_loss: 1.4258 -
val_accuracy: 0.4948
Epoch 23/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.3775 - accuracy: 0.5077 - val_loss: 1.4029 -
val_accuracy: 0.5053
Epoch 24/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.3650 - accuracy: 0.5115 - val_loss: 1.4012 -
val_accuracy: 0.4968

Epoch 25/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.3542 - accuracy: 0.5147 - val_loss: 1.3649 -
val_accuracy: 0.5183
Epoch 26/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.3414 - accuracy: 0.5200 - val_loss: 1.4072 -
val_accuracy: 0.5006
Epoch 27/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.3337 - accuracy: 0.5225 - val_loss: 1.3618 -
val_accuracy: 0.5112
Epoch 28/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.3222 - accuracy: 0.5262 - val_loss: 1.3390 -
val_accuracy: 0.5252
Epoch 29/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.3153 - accuracy: 0.5270 - val_loss: 1.3671 -
val_accuracy: 0.5083
Epoch 30/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.3045 - accuracy: 0.5333 - val_loss: 1.3487 -
val_accuracy: 0.5244
Epoch 31/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.2917 - accuracy: 0.5374 - val_loss: 1.3268 -
val_accuracy: 0.5260
Epoch 32/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.2863 - accuracy: 0.5373 - val_loss: 1.3364 -
val_accuracy: 0.5223
Epoch 33/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.2750 - accuracy: 0.5451 - val_loss: 1.3153 -
val_accuracy: 0.5303
Epoch 34/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.2608 - accuracy: 0.5492 - val_loss: 1.3283 -
val_accuracy: 0.5331
Epoch 35/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.2570 - accuracy: 0.5492 - val_loss: 1.3215 -
val_accuracy: 0.5339
Epoch 36/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.2461 - accuracy: 0.5546 - val_loss: 1.3488 -
val_accuracy: 0.5282
Epoch 37/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.2377 - accuracy: 0.5560 - val_loss: 1.3261 -
val_accuracy: 0.5291
Epoch 38/100
1563/1563 [=====] - 7s 4ms/step - loss: 1.2298 - accuracy: 0.5589 - val_loss: 1.3143 -
val_accuracy: 0.5363
Epoch 39/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.2202 - accuracy: 0.5616 - val_loss: 1.2962 -
val_accuracy: 0.5396
Epoch 40/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.2152 - accuracy: 0.5643 - val_loss: 1.2800 -
val_accuracy: 0.5435
Epoch 41/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.2044 - accuracy: 0.5693 - val_loss: 1.2747 -
val_accuracy: 0.5433
Epoch 42/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1940 - accuracy: 0.5706 - val_loss: 1.2688 -
val_accuracy: 0.5486
Epoch 43/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1879 - accuracy: 0.5750 - val_loss: 1.2752 -
val_accuracy: 0.5435
Epoch 44/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1789 - accuracy: 0.5760 - val_loss: 1.2800 -
val_accuracy: 0.5438
Epoch 45/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1684 - accuracy: 0.5802 - val_loss: 1.2705 -
val_accuracy: 0.5485
Epoch 46/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1616 - accuracy: 0.5824 - val_loss: 1.2643 -
val_accuracy: 0.5514
Epoch 47/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1537 - accuracy: 0.5849 - val_loss: 1.2833 -
val_accuracy: 0.5521
Epoch 48/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1469 - accuracy: 0.5895 - val_loss: 1.2533 -
val_accuracy: 0.5533

Epoch 49/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1404 - accuracy: 0.5912 - val_loss: 1.2445 -
val_accuracy: 0.5547
Epoch 50/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1287 - accuracy: 0.5926 - val_loss: 1.2692 -
val_accuracy: 0.5475
Epoch 51/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1188 - accuracy: 0.5977 - val_loss: 1.2921 -
val_accuracy: 0.5466
Epoch 52/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1141 - accuracy: 0.6014 - val_loss: 1.2673 -
val_accuracy: 0.5509
Epoch 53/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.1053 - accuracy: 0.6039 - val_loss: 1.2723 -
val_accuracy: 0.5441
Epoch 54/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0983 - accuracy: 0.6054 - val_loss: 1.2756 -
val_accuracy: 0.5521
Epoch 55/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0883 - accuracy: 0.6074 - val_loss: 1.2836 -
val_accuracy: 0.5482
Epoch 56/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0842 - accuracy: 0.6101 - val_loss: 1.2478 -
val_accuracy: 0.5588
Epoch 57/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0713 - accuracy: 0.6141 - val_loss: 1.2687 -
val_accuracy: 0.5521
Epoch 58/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0678 - accuracy: 0.6165 - val_loss: 1.3101 -
val_accuracy: 0.5485
Epoch 59/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0589 - accuracy: 0.6202 - val_loss: 1.2443 -
val_accuracy: 0.5612
Epoch 60/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0518 - accuracy: 0.6214 - val_loss: 1.2298 -
val_accuracy: 0.5632
Epoch 61/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0450 - accuracy: 0.6245 - val_loss: 1.2494 -
val_accuracy: 0.5581
Epoch 62/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0360 - accuracy: 0.6272 - val_loss: 1.2295 -
val_accuracy: 0.5707
Epoch 63/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0327 - accuracy: 0.6267 - val_loss: 1.2416 -
val_accuracy: 0.5589
Epoch 64/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0248 - accuracy: 0.6320 - val_loss: 1.2373 -
val_accuracy: 0.5635
Epoch 65/100
1563/1563 [=====] - 7s 4ms/step - loss: 1.0131 - accuracy: 0.6350 - val_loss: 1.2770 -
val_accuracy: 0.5542
Epoch 66/100
1563/1563 [=====] - 6s 4ms/step - loss: 1.0081 - accuracy: 0.6384 - val_loss: 1.2554 -
val_accuracy: 0.5589
Epoch 67/100
1563/1563 [=====] - 7s 4ms/step - loss: 1.0031 - accuracy: 0.6384 - val_loss: 1.2515 -
val_accuracy: 0.5612
Epoch 68/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.9943 - accuracy: 0.6394 - val_loss: 1.2339 -
val_accuracy: 0.5655
Epoch 69/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.9854 - accuracy: 0.6444 - val_loss: 1.2294 -
val_accuracy: 0.5623
Epoch 70/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.9769 - accuracy: 0.6482 - val_loss: 1.2202 -
val_accuracy: 0.5711
Epoch 71/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.9731 - accuracy: 0.6487 - val_loss: 1.2668 -
val_accuracy: 0.5580
Epoch 72/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.9621 - accuracy: 0.6519 - val_loss: 1.2332 -
val_accuracy: 0.5663

Epoch 73/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.9594 - accuracy: 0.6543 - val_loss: 1.2341 - val_accuracy: 0.5685
Epoch 74/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.9479 - accuracy: 0.6582 - val_loss: 1.2146 - val_accuracy: 0.5707
Epoch 75/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.9436 - accuracy: 0.6611 - val_loss: 1.2293 - val_accuracy: 0.5714
Epoch 76/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.9360 - accuracy: 0.6630 - val_loss: 1.2413 - val_accuracy: 0.5643
Epoch 77/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.9261 - accuracy: 0.6662 - val_loss: 1.2519 - val_accuracy: 0.5675
Epoch 78/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.9207 - accuracy: 0.6679 - val_loss: 1.2295 - val_accuracy: 0.5695
Epoch 79/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.9146 - accuracy: 0.6709 - val_loss: 1.2481 - val_accuracy: 0.5728
Epoch 80/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.9099 - accuracy: 0.6722 - val_loss: 1.2362 - val_accuracy: 0.5711
Epoch 81/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.9024 - accuracy: 0.6734 - val_loss: 1.2466 - val_accuracy: 0.5671
Epoch 82/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.8911 - accuracy: 0.6784 - val_loss: 1.2453 - val_accuracy: 0.5691
Epoch 83/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.8846 - accuracy: 0.6824 - val_loss: 1.2538 - val_accuracy: 0.5730
Epoch 84/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.8809 - accuracy: 0.6828 - val_loss: 1.2253 - val_accuracy: 0.5731
Epoch 85/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.8722 - accuracy: 0.6842 - val_loss: 1.2223 - val_accuracy: 0.5758
Epoch 86/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.8659 - accuracy: 0.6871 - val_loss: 1.2660 - val_accuracy: 0.5714
Epoch 87/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.8586 - accuracy: 0.6900 - val_loss: 1.2693 - val_accuracy: 0.5726
Epoch 88/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.8542 - accuracy: 0.6931 - val_loss: 1.2688 - val_accuracy: 0.5674
Epoch 89/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.8459 - accuracy: 0.6960 - val_loss: 1.2368 - val_accuracy: 0.5770
Epoch 90/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.8389 - accuracy: 0.6949 - val_loss: 1.2310 - val_accuracy: 0.5728
Epoch 91/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.8366 - accuracy: 0.6988 - val_loss: 1.2885 - val_accuracy: 0.5680
Epoch 92/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.8222 - accuracy: 0.7033 - val_loss: 1.3369 - val_accuracy: 0.5488
Epoch 93/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.8185 - accuracy: 0.7036 - val_loss: 1.2760 - val_accuracy: 0.5707
Epoch 94/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.8104 - accuracy: 0.7082 - val_loss: 1.2633 - val_accuracy: 0.5788
Epoch 95/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.7998 - accuracy: 0.7120 - val_loss: 1.2728 - val_accuracy: 0.5777
Epoch 96/100
1563/1563 [=====] - 7s 4ms/step - loss: 0.8006 - accuracy: 0.7099 - val_loss: 1.2274 - val_accuracy: 0.5806

```
Epoch 97/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.7951 - accuracy: 0.7129 - val_loss: 1.2444 -
val_accuracy: 0.5706
Epoch 98/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.7836 - accuracy: 0.7166 - val_loss: 1.3236 -
val_accuracy: 0.5546
Epoch 99/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.7801 - accuracy: 0.7194 - val_loss: 1.2561 -
val_accuracy: 0.5678
Epoch 100/100
1563/1563 [=====] - 6s 4ms/step - loss: 0.7721 - accuracy: 0.7225 - val_loss: 1.2448 -
val_accuracy: 0.5798
```

Out[]: <keras.callbacks.History at 0x7f514478e250>

Question 4:

Complete the following table with your final design (you may add more rows for the # neurons (layer1) etc. to detail how many neurons you have in each hidden layer). Likewise you may replace the lr, momentum etc rows with parameters more appropriate to the optimizer that you have chosen. (3 MARKS)

Hyperparameter	What I used	Why?	Optimizer
SGD	Performs similar to or better than other optimizers like Adam in our experiments.		
# of hidden layers	2	Add layer to represent more complex relationship	
# neurons(layer1)	1024	Balance between layers before and after	
Hid layer1 activation	Sigmoid	Nonlinear relationship, better performance than ReLU based on our tests	
# neurons(layer2)	1024	Balance between layers before and after	
Hid layer2 activation	Sigmoid	Nonlinear relationship	
# of output neurons	10	Number of classes	
Output activation	Softmax	Convert values to probability distribution over classes	
learning rate	0.1	Too high it does not converge, too low it is too slow	
momentum	0.1	Some momentum to prevent getting stuck, too high would converge	
decay	None	Model already underfitting, would not improve getting stuck in local optima	
loss	categorical cross entropy	task is the same, multi-class classification	

FOR GRADER:

TABLE: ____ / 3

CODE: ____ / 5

TOTAL: ____ / 8

Question 5

What is the final training and validation accuracy that you obtained after 100 epochs. Is there considerable improvement over Section 3.2? Are there still signs of underfitting or overfitting? Explain your answer (5 MARKS)

Training: 0.72, validation: 0.58. An improvement from 3.2. There is still a large sign of overfitting, since the validation accuracy is significantly lower than the training accuracy. There is slightly less underfitting with the training accuracy improving.

FOR GRADER: ____ / 5

Question 6

Write a short reflection on the practical difficulties of using a dense MLP to classify images in the CIFAR-10 datasets. (3 MARKS)

Training takes a very long time even with a GPU, and optimizing hyperparameters is not straightforward. The amount of time spent optimizing does not directly correlate with better performance. Optimizing does not give significantly better results. As such there are some considerations when choosing to optimize hyperparameters. For example, if faster results are needed, we should not bother optimizing the hyperparameters too much as the improvement is not significant. However, if obtaining the best possible results is our goal and we do not have time considerations, we can choose to optimize the hyperparameters.

FOR GRADER: ____ / 3

4. Creating a CNN for the MNIST Dataset

In this section we will now create a convolutional neural network (CNN) to classify images in the MNIST dataset that we used in the previous lab. Let's go through each part to see how to do this.

4.1 Loading the MNIST Dataset

As always we will load the MNIST dataset, scale the inputs to between 0 and 1, and convert the Y labels to one-hot vectors. However unlike before we will not flatten the 28x28 image to a 784 element vector, since CNNs can inherently handle 2D data.

```
In [ ]: from tensorflow.keras.datasets import mnist
        from tensorflow.keras.utils import to_categorical

        def load_mnist():
            (train_x, train_y), (test_x, test_y) = mnist.load_data()
            train_x = train_x.reshape(train_x.shape[0], 28, 28, 1)
            test_x = test_x.reshape(test_x.shape[0], 28, 28, 1)

            train_x = train_x.astype('float32')
            test_x = test_x.astype('float32')

            train_x /= 255.0
            test_x /= 255.0

            train_y = to_categorical(train_y, 10)
            test_y = to_categorical(test_y, 10)

            return (train_x, train_y), (test_x, test_y)
```

4.2 Building the CNN

We will now build the CNN. Unlike before we will create a function to produce the CNN. We will also look at how to save and load Keras models using "checkpoints", particularly "ModelCheckpoint" that saves the model each epoch.

Let's begin by creating the model. We call `os.path.exists` to see if a model file exists, and call "load_model" if it does. Otherwise we create a new model.

```
In [ ]: # Load_model Loads a model from a h5 file.
        from tensorflow.keras.models import Sequential, load_model
        from tensorflow.keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPooling2D
        import os

        MODEL_NAME = 'mnist-cnn.h5'

        def buildmodel(model_name):
            if os.path.exists(model_name):
                model = load_model(model_name)
            else:
                model = Sequential()
                model.add(Conv2D(32, kernel_size=(5,5),
                                activation='relu',
                                input_shape=(28, 28, 1), padding='same')) # Question 7

                model.add(MaxPooling2D(pool_size=(2,2), strides=2)) # Question 8
                model.add(Conv2D(64, kernel_size=(5,5), activation='relu'))
                model.add(Conv2D(128, kernel_size=(5,5), activation='relu'))
                model.add(Conv2D(64, kernel_size=(5,5), activation='relu'))
                model.add(MaxPooling2D(pool_size=(2,2), strides=2))
                model.add(Flatten()) # Question 9
                model.add(Dense(1024, activation='relu'))
                model.add(Dropout(0.1))
                model.add(Dense(10, activation='softmax'))

            return model
```

Question 7

The first layer in our CNN is a 2D convolution kernel, shown here:

```
model.add(Conv2D(32, kernel_size=(5,5),
activation='relu',
input_shape=(28, 28, 1), padding='same')) # Question 7
```

Why is the input_shape set to (28, 28, 1)? What does this mean? What does "padding = 'same'" mean? (4 MARKS)

It is the size of a single input data, with 28 rows, 28 columns, and 1 channel. Same padding will lead to the output size being the same as the input size.

FOR GRADER: ____ / 4

Question 8

The second layer is the MaxPooling2D layer shown below:

```
model.add(MaxPooling2D(pool_size=(2,2), strides=2)) # Question 8
```

What other types of pooling layers are available? What does 'strides = 2' mean? (3 MARKS)

There are also Average pooling, global average pooling, and global max pooling layers. Strides = 2 means the filter moves every 2 steps along each dimension.

FOR GRADER: ____ / 3

Question 9

What does the "Flatten" layer here do? Why is it needed?

```
model.add(Flatten()) # Question 9
```

To input into the Dense layer, the input has to be a single vector, so the 2D output from the convolutional layers has to be flattened first.

FOR GRADER: ____ / 2

4.3 Training the CNN

Let's now train the CNN. In this example we introduce the idea of a "callback", which is a routine that Keras calls at the end of each epoch. Specifically we look at two callbacks:

1. ModelCheckpoint: When called, Keras saves the model to the specified filename.
2. EarlyStopping: When called, Keras checks if it should stop the training prematurely.

Let's look at the code to see how training is done, and how callbacks are used.

```
In [ ]: from tensorflow.keras.optimizers import SGD
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

def train(model, train_x, train_y, epochs, test_x, test_y, model_name):

    model.compile(optimizer=SGD(lr=0.01, momentum=0.7),
                  loss='categorical_crossentropy', metrics=['accuracy'])

    savemodel = ModelCheckpoint(model_name)
```

```

stopmodel = EarlyStopping(min_delta=0.001, patience=10) # Question 10

print("Starting training.")

model.fit(x=train_x, y=train_y, batch_size=32,
          validation_data=(test_x, test_y), shuffle=True,
          epochs=epochs,
          callbacks=[savemodel, stopmodel])

print("Done. Now evaluating.")
loss, acc = model.evaluate(x=test_x, y=test_y)
print("Test accuracy: %3.2f, loss: %3.2f"%(acc, loss))

```

Notice that there isn't very much that is unusual going on; we compile the model with our loss function and optimizer, then call fit, and finally evaluate to look at the final accuracy for the test set. The only thing unusual is the "callbacks" parameter here in the fit function call

```

model.fit(x=train_x, y=train_y, batch_size=32,
          validation_data=(test_x, test_y), shuffle=True,
          epochs=epochs,
          callbacks=[savemodel, stopmodel])

```

Question 10.

What do the min_delta and patience parameters do in the EarlyStopping callback, as shown below? (2 MARKS)

```
stopmodel = EarlyStopping(min_delta=0.001, patience=10) # Question 10
```

* **min_delta** is the minimum change to qualify as an improvement, and **patience** is the number of epochs of no improvement before training will be stopped.*

FOR GRADER: ____ / 2

4.4 Putting it together.

Now let's run the code and see how it goes (Note: To save time we are training for only 5 epochs; we should train much longer to get much better results):

```

In [ ]: (train_x, train_y), (test_x, test_y) = load_mnist()
        model = buildmodel(MODEL_NAME)
        train(model, train_x, train_y, 5, test_x, test_y, MODEL_NAME)

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11493376/11490434 [=====] - 0s 0us/step

11501568/11490434 [=====] - 0s 0us/step

/usr/local/lib/python3.7/dist-packages/keras/optimizer_v2/gradient_descent.py:102: UserWarning: The `lr` argument is deprecated, use `learning\_rate` instead.

```
super(SGD, self).__init__(name, **kwargs)
```

```

Starting training.
Epoch 1/5
1875/1875 [=====] - 19s 5ms/step - loss: 0.3645 - accuracy: 0.8829 - val_loss: 0.0776
- val_accuracy: 0.9769
Epoch 2/5
1875/1875 [=====] - 10s 5ms/step - loss: 0.0720 - accuracy: 0.9776 - val_loss: 0.0594
- val_accuracy: 0.9822
Epoch 3/5
1875/1875 [=====] - 10s 5ms/step - loss: 0.0477 - accuracy: 0.9853 - val_loss: 0.0425
- val_accuracy: 0.9872
Epoch 4/5
1875/1875 [=====] - 10s 5ms/step - loss: 0.0355 - accuracy: 0.9890 - val_loss: 0.0358
- val_accuracy: 0.9895
Epoch 5/5
1875/1875 [=====] - 9s 5ms/step - loss: 0.0271 - accuracy: 0.9915 - val_loss: 0.0450 -
val_accuracy: 0.9863
Done. Now evaluating.
313/313 [=====] - 1s 4ms/step - loss: 0.0450 - accuracy: 0.9863
Test accuracy: 0.99, loss: 0.04

```

Question 11.

Compare the relative advantages and disadvantages of CNN vs. the Dense MLP that you build in sections 3.2 and 3.3. What makes CNNs better (or worse)? (3 MARKS)

***CNNs take advantage of parameter sharing between parts of the image that are close by, as well as the sparsity of connections between each layer allowing for a more efficient network. While Dense MLPs will find the relationship between all parts of a previous layer and the next layer, CNNs will find more relevant relationships, saving on resources at the same time. ***

FOR GRADER: ____ / 3

5. Creating a CNN for the CIFAR-10 Dataset

Now comes the fun part: Using the example above for creating a CNN for the MNIST dataset, now create a CNN in the box below for the MNIST-10 dataset. At the end of each epoch save the model to a file called "cifar.hd5" (note: the .hd5 is added automatically for you).

Question 12.

Summarize your design in the table below (the actual coding cell comes after this):

Hyperparameter	What I used	Why?	Optimizer	SGD	Works well
Input shape	(32, 32, 3)	Input size of a picture	First layer	Conv2D	Picture data using convolution
Second layer	MaxPooling2D	Aggregate local data using max pooling, common practice	Add more layers	Conv2D x3, MaxPool, Dropout, Flatten	Model complex relationship with multiple layers
if needed	Dropout	Added to prevent overfitting	Dense layer	1024	Output eventual 10 classes

FOR GRADER:

TABLE: ____ / 3

CODE: ____ / 7

TOTAL: ____ / 10

TOTAL: ____ / 55

In []:

```

"""
Write your code for your CNN for the CIFAR-10 dataset here.

Note: train_x, train_y, test_x, test_y were changed when we called
load_mnist in the previous section. You will now need to call load_cifar10
again.

```

```

"""
MODEL_NAME_CIFAR = 'cifar.hd5'

(train_x, train_y), (test_x, test_y) = load_cifar10()
train_x = train_x.reshape(train_x.shape[0], 32, 32, 3)
test_x = test_x.reshape(test_x.shape[0], 32, 32, 3)

def buildmodel(model_name):
    model = Sequential()
    model.add(Conv2D(32, kernel_size=(5,5),
        activation='relu',
        input_shape=(32, 32, 3), padding='same')) # Question 7

    model.add(MaxPooling2D(pool_size=(2,2), strides=2)) # Question 8
    model.add(Dropout(0.5))

    model.add(Conv2D(64, kernel_size=(5,5), activation='relu'))
    model.add(Conv2D(128, kernel_size=(5,5), activation='relu'))
    model.add(Conv2D(64, kernel_size=(5,5), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2), strides=2))
    model.add(Dropout(0.5))

    model.add(Flatten()) # Question 9
    model.add(Dense(512, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(10, activation='softmax'))

    return model

model = buildmodel(MODEL_NAME_CIFAR)
train(model, train_x, train_y, 50, test_x, test_y, MODEL_NAME_CIFAR)

```

```

/usr/local/lib/python3.7/dist-packages/keras/optimizer_v2/gradient_descent.py:102: UserWarning: The `lr` argument is deprecated, use `learning_rate` instead.
  super(SGD, self).__init__(name, **kwargs)

```


Starting training.

Epoch 1/50

1563/1563 [=====] - 10s 6ms/step - loss: 1.9996 - accuracy: 0.2398 - val_loss: 1.6990
- val_accuracy: 0.3933

Epoch 2/50

1563/1563 [=====] - 10s 6ms/step - loss: 1.5985 - accuracy: 0.4128 - val_loss: 1.4512
- val_accuracy: 0.4639

Epoch 3/50

1563/1563 [=====] - 10s 7ms/step - loss: 1.4263 - accuracy: 0.4838 - val_loss: 1.4395
- val_accuracy: 0.4949

Epoch 4/50

1563/1563 [=====] - 10s 7ms/step - loss: 1.3135 - accuracy: 0.5308 - val_loss: 1.2016
- val_accuracy: 0.5717

Epoch 5/50

1563/1563 [=====] - 10s 6ms/step - loss: 1.2283 - accuracy: 0.5663 - val_loss: 1.1580
- val_accuracy: 0.5963

Epoch 6/50

1563/1563 [=====] - 10s 7ms/step - loss: 1.1461 - accuracy: 0.5980 - val_loss: 1.0391
- val_accuracy: 0.6430

Epoch 7/50

1563/1563 [=====] - 10s 6ms/step - loss: 1.0841 - accuracy: 0.6217 - val_loss: 1.1124
- val_accuracy: 0.6219

Epoch 8/50

1563/1563 [=====] - 10s 7ms/step - loss: 1.0291 - accuracy: 0.6406 - val_loss: 0.9630
- val_accuracy: 0.6661

Epoch 9/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.9758 - accuracy: 0.6615 - val_loss: 0.9853
- val_accuracy: 0.6625

Epoch 10/50

1563/1563 [=====] - 10s 7ms/step - loss: 0.9325 - accuracy: 0.6770 - val_loss: 0.9851
- val_accuracy: 0.6559

Epoch 11/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.8983 - accuracy: 0.6885 - val_loss: 0.8918
- val_accuracy: 0.6946

Epoch 12/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.8677 - accuracy: 0.6992 - val_loss: 0.8165
- val_accuracy: 0.7214

Epoch 13/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.8373 - accuracy: 0.7107 - val_loss: 0.8011
- val_accuracy: 0.7251

Epoch 14/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.8087 - accuracy: 0.7234 - val_loss: 0.7972
- val_accuracy: 0.7307

Epoch 15/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.7871 - accuracy: 0.7294 - val_loss: 0.8083
- val_accuracy: 0.7279

Epoch 16/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.7624 - accuracy: 0.7374 - val_loss: 0.7642
- val_accuracy: 0.7375

Epoch 17/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.7386 - accuracy: 0.7435 - val_loss: 0.7907
- val_accuracy: 0.7361

Epoch 18/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.7246 - accuracy: 0.7501 - val_loss: 0.7392
- val_accuracy: 0.7519

Epoch 19/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.7036 - accuracy: 0.7576 - val_loss: 0.7598
- val_accuracy: 0.7398

Epoch 20/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.6895 - accuracy: 0.7627 - val_loss: 0.7278
- val_accuracy: 0.7543

Epoch 21/50

1563/1563 [=====] - 10s 6ms/step - loss: 0.6742 - accuracy: 0.7665 - val_loss: 0.7889
- val_accuracy: 0.7316

Epoch 22/50

1563/1563 [=====] - 10s 7ms/step - loss: 0.6634 - accuracy: 0.7724 - val_loss: 0.7526
- val_accuracy: 0.7489

Epoch 23/50

1563/1563 [=====] - 11s 7ms/step - loss: 0.6462 - accuracy: 0.7766 - val_loss: 0.7120
- val_accuracy: 0.7587

Epoch 24/50

1563/1563 [=====] - 10s 7ms/step - loss: 0.6351 - accuracy: 0.7802 - val_loss: 0.7700

- val_accuracy: 0.7378
Epoch 25/50
1563/1563 [=====] - 10s 7ms/step - loss: 0.6273 - accuracy: 0.7849 - val_loss: 0.7426
- val_accuracy: 0.7545
Epoch 26/50
1563/1563 [=====] - 10s 6ms/step - loss: 0.6078 - accuracy: 0.7887 - val_loss: 0.7095
- val_accuracy: 0.7582
Epoch 27/50
1563/1563 [=====] - 10s 6ms/step - loss: 0.5992 - accuracy: 0.7925 - val_loss: 0.7007
- val_accuracy: 0.7667
Epoch 28/50
1563/1563 [=====] - 10s 6ms/step - loss: 0.5911 - accuracy: 0.7949 - val_loss: 0.7179
- val_accuracy: 0.7535
Epoch 29/50
1563/1563 [=====] - 10s 6ms/step - loss: 0.5790 - accuracy: 0.8001 - val_loss: 0.7211
- val_accuracy: 0.7595
Epoch 30/50
1563/1563 [=====] - 10s 6ms/step - loss: 0.5754 - accuracy: 0.8029 - val_loss: 0.6945
- val_accuracy: 0.7692
Epoch 31/50
1563/1563 [=====] - 10s 7ms/step - loss: 0.5603 - accuracy: 0.8064 - val_loss: 0.7240
- val_accuracy: 0.7637
Epoch 32/50
1563/1563 [=====] - 10s 6ms/step - loss: 0.5554 - accuracy: 0.8058 - val_loss: 0.7056
- val_accuracy: 0.7658
Epoch 33/50
1563/1563 [=====] - 10s 7ms/step - loss: 0.5468 - accuracy: 0.8103 - val_loss: 0.6999
- val_accuracy: 0.7661
Epoch 34/50
1563/1563 [=====] - 10s 6ms/step - loss: 0.5435 - accuracy: 0.8127 - val_loss: 0.6912
- val_accuracy: 0.7703
Epoch 35/50
1563/1563 [=====] - 10s 7ms/step - loss: 0.5272 - accuracy: 0.8173 - val_loss: 0.6901
- val_accuracy: 0.7685
Epoch 36/50
1563/1563 [=====] - 10s 6ms/step - loss: 0.5270 - accuracy: 0.8185 - val_loss: 0.6968
- val_accuracy: 0.7658
Epoch 37/50
1563/1563 [=====] - 10s 6ms/step - loss: 0.5171 - accuracy: 0.8230 - val_loss: 0.6889
- val_accuracy: 0.7710
Epoch 38/50
1563/1563 [=====] - 10s 7ms/step - loss: 0.5145 - accuracy: 0.8224 - val_loss: 0.6859
- val_accuracy: 0.7724
Epoch 39/50
1563/1563 [=====] - 11s 7ms/step - loss: 0.5069 - accuracy: 0.8242 - val_loss: 0.7413
- val_accuracy: 0.7571
Epoch 40/50
1563/1563 [=====] - 10s 7ms/step - loss: 0.4989 - accuracy: 0.8268 - val_loss: 0.7143
- val_accuracy: 0.7630
Epoch 41/50
1563/1563 [=====] - 10s 7ms/step - loss: 0.4930 - accuracy: 0.8293 - val_loss: 0.6927
- val_accuracy: 0.7711
Epoch 42/50
1563/1563 [=====] - 10s 7ms/step - loss: 0.4872 - accuracy: 0.8303 - val_loss: 0.7030
- val_accuracy: 0.7668
Epoch 43/50
1563/1563 [=====] - 11s 7ms/step - loss: 0.4879 - accuracy: 0.8320 - val_loss: 0.6925
- val_accuracy: 0.7713
Epoch 44/50
1563/1563 [=====] - 10s 6ms/step - loss: 0.4794 - accuracy: 0.8333 - val_loss: 0.6866
- val_accuracy: 0.7767
Epoch 45/50
1563/1563 [=====] - 10s 7ms/step - loss: 0.4763 - accuracy: 0.8346 - val_loss: 0.7094
- val_accuracy: 0.7667
Epoch 46/50
1563/1563 [=====] - 10s 7ms/step - loss: 0.4717 - accuracy: 0.8367 - val_loss: 0.7060
- val_accuracy: 0.7686
Epoch 47/50
1563/1563 [=====] - 11s 7ms/step - loss: 0.4669 - accuracy: 0.8396 - val_loss: 0.7234
- val_accuracy: 0.7674
Epoch 48/50
1563/1563 [=====] - 10s 6ms/step - loss: 0.4586 - accuracy: 0.8401 - val_loss: 0.7097

- val_accuracy: 0.7715

Done. Now evaluating.

313/313 [=====] - 1s 4ms/step - loss: 0.7097 - accuracy: 0.7715

Test accuracy: 0.77, loss: 0.71